Samsonova Darya, Khazratova Guzel, Lu Ke, Shao Shiyun, Hao Xiaohua

A RAG TRACE PLATFORM FOR FAULT LOCALIZATION: PIPELINE, CONTROLLED-FAILURE METRICS, AND CROSS-LINGUAL PORTABILITY TO RUSSIAN

Abstract. Diagnosing why a retrieval-augmented generation (RAG) system fails requires visibility into every stage of the pipeline, not just the final answer. We present a modular framework that ingests two structurally incompatible datasets through a unified trace schema and a modular processing pipeline. To validate this system, we construct a controlled-failure benchmark by injecting nine fault types and compute an eleven-metric bundle per trace, applying a deterministic, precedence-based rule cascade for diagnosis. We then evaluate the entire pipeline's portability—including the controlled-failure evaluation itself—by reproducing the study on Russian. On the English validation set, the system reaches 78.33% coverage with 64.58% supported exact accuracy, with clearly documented blind spots. The Russian reproduction shows a similar qualitative signature, supporting the claim that structure-sensitive signals transfer across languages while lexical generation metrics remain more language-dependent.

§1. Introduction

A retrieval-augmented generation (RAG) system can fail for many reasons: the knowledge base may lack the answer, chunking may split the evidence, retrieval may miss the relevant passage, context construction may drop it, or the generator may hallucinate. Diagnosing which stage failed requires a record of what happened at every stage, not just the final answer [1]. The two datasets available for this project, however, expose only endpoints (a question, some context, an answer) and hide the intermediate process, and they do so in two incompatible formats: an English HotpotQA-style multi-document set [2] and a Russian long-context set built around a single concatenated passage.

This project addresses that gap by building a fully instrumented RAG Trace Platform that turns heterogeneous raw data into schema-valid, stage-by-stage traces. We then validate trace quality and construct a controlled-failure benchmark by injecting nine known fault types into healthy traces. A deterministic rule cascade over an eleven-metric bundle turns that metric bundle into a fault-family diagnosis, an abstention, or an input-error label. Finally, a cross-lingual portability study reruns the entire pipeline on Russian to test whether the method transfers or was implicitly tuned to English.

Portability is a central theme: a fault-diagnosis framework is only useful in practice if it transfers beyond the language and domain it was developed on. We report our results, including known limitations, at the level of evidence they actually support.

§2. The RAG Trace Platform

The English data provides, per question, a set of ten candidate documents—two gold documents plus eight distractors—together with structured supporting_facts identifying the gold sentences [2]. The Russian data provides a question, a single long concatenated context blob (a gold passage embedded among distractor text), and the gold passage itself. The two are structurally unrelated.

To make every downstream stage dataset-agnostic, we defined a single source_sample schema and wrote one adapter per dataset that maps the raw fields onto it. Both adapters normalize gold evidence into a uniform representation: each gold item always carries a document_id and the gold_text, plus a sentence_index for English (sentence-structured evidence) or character offsets for Russian (substring-located evidence). The schema is defined formally in source_sample.schema.json, and every adapter output is validated against it, so that supporting a new data source requires only a new adapter, never a change to the pipeline.

The pipeline is a sequence of independent stages, each a standalone script that reads the previous stage's output and appends only its own fields, so a finished trace accumulates the entire processing history. Source documents are split by a sentence-aware recursive chunker using a tiktoken tokenizer (target 400 tokens, 80-token overlap). Each chunk records its character span and token count; the stage also computes, for every gold evidence item, which chunks cover it and whether it survived intact—a gold-preservation signal used later to define "healthy" traces.

Retrieval is performed per sample: each question is retrieved against its own candidate pool rather than a global index, mirroring how both datasets are constructed and guaranteeing the gold passage is in-pool. Two interchangeable retrievers sit behind one interface: a sparse BM25 retriever [5] and a dense

Key words and phrases: retrieval-augmented generation and fault diagnosis and RAG evaluation and controlled failure injection and cross-lingual portability and BM25 and dense retrieval.

retriever using a multilingual sentence-transformer (MiniLM) with cosine similarity [3, 4], following the dense-retrieval paradigm popularized for open-domain question answering [6]. Each retriever returns ranked chunks with scores, and the stage reports Recall@k and Hit@k against the gold chunks.

The retrieved top-k chunks are assembled into the final context by rank ordering, de-duplication, and a token budget (3000 tokens). The stage records which chunks were selected, dropped, or de-duplicated, whether truncation occurred, and whether gold evidence survived into the final context. The final context and query are then combined by a versioned, grounded prompt (grounded_v1, instructing the model to answer only from the supplied context and to state explicitly when the context is insufficient) and passed to the generator at temperature = 0. The generator is a swappable LLM backend; reported runs use GLM-4.5-Flash [7]. The stage records the final prompt, answer, token counts, latency, and status, with retry-with-backoff and resumable execution to tolerate rate limits. The output of this stage is a complete trace.

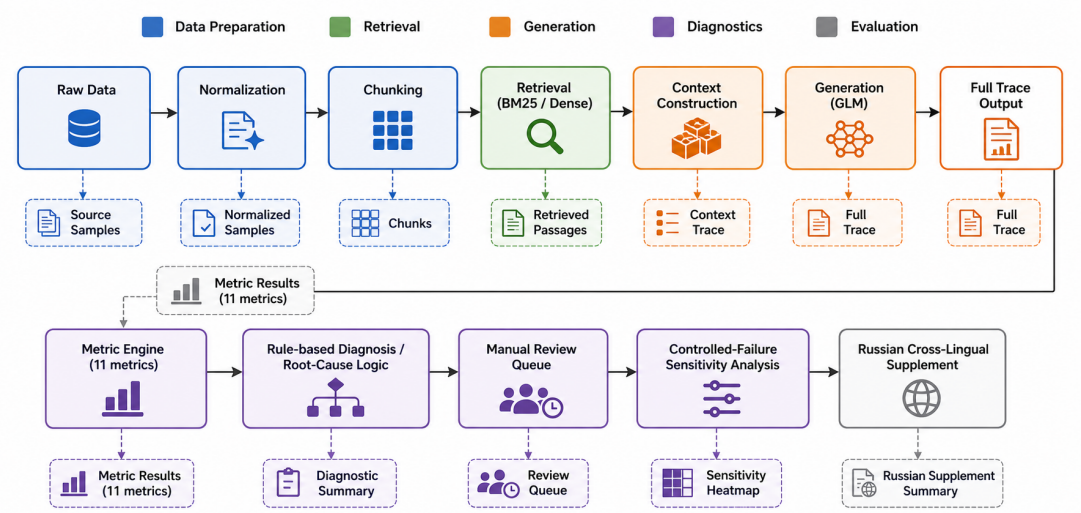


Figure 1. RAG Trace Platform pipeline: adapters normalize two incompatible datasets into a unified schema; chunking, retrieval, context construction, and generation each append fields to a single accumulating trace record.

Four principles govern the platform. Per-sample retrieval keeps the retrieval task faithful to the datasets and ensures gold is retrievable. Configuration transparency means every stage serializes its configuration (chunk size, tokenizer, retriever, top-k, embedding model, prompt version, model, temperature) into the trace, and any genuinely unknown configuration is marked as such rather than fabricated. The two-system boundary keeps the platform strictly a producer of traces—it never scores them—so that fault-injection ground truth cannot leak into the evaluator. Finally, both the source samples and the complete traces are governed by JSON schemas (source_sample.schema.json, full_trace.schema.json) and validated, giving downstream stages a stable contract.

Each complete trace is a single JSON line conforming to full_trace.schema.json: identity fields (trace_id, source_record_id, pipeline_id, language), the query and reference answer, gold evidence with per-chunk coverage, the chunks, the retrieval result and its evaluation, the context-construction record, the generation record, accumulated pipeline configuration, an injected_fault slot (null for natural traces), and runtime status. A distinct pipeline_id per configuration allows the same question, processed by different pipelines, to be grouped for pipeline-level analysis while remaining individually identifiable by trace_id.

§3. Data Quality and Controlled-Failure Construction

At this stage, the initial RAG traces produced by the platform were analyzed. A trace contains the full record of one RAG system run: user query, retrieved documents, context, and final model response. The main goal was to identify correct ("healthy") traces suitable for further experiments.

A validation procedure checked file structure, the presence of required components, and the presence of supporting documents (gold evidence). The analysis was performed for both retrieval methods, BM25 and Dense Retrieval, with 500 traces processed for each.

More than 99% of traces had a correct structure; however, a complete set of supporting documents was present in only about 58% of cases (Table 1). Additional data filtering was therefore applied before

Metric	BM25	Dense Retrieval
Usable traces	496 (99.2%)	497 (99.4%)
At least one gold evidence present	487 (97.4%)	482 (96.4%)
Full gold evidence present	293 (58.6%)	291 (58.2%)

Table 1. Data quality check results on the English trace set (500 traces per retriever).

further experiments: 293 healthy BM25 traces and 291 healthy Dense Retrieval traces were retained as the basis for controlled-failure injection.

A controlled error-generation mechanism was developed to simulate realistic RAG failure modes. Nine fault types were implemented, covering the main stages of system operation: document retrieval, context formation (chunking), answer generation, as well as malformed queries and uncertain/out-of-scope cases. Based on the healthy traces, 180 controlled-error examples were generated for BM25 and 180 for Dense Retrieval, each carrying a ground-truth injected_fault label used later to evaluate diagnostic methods.

To verify the correctness of the fault-injection framework itself, an annotator manually classified a sample of the generated examples, and the results were compared against the expected labels.

Method	Matches	Accuracy
BM25	17 out of 18	94.4%
Dense Retrieval	14 out of 17	82.4%

Table 2. Manual validation of the controlled fault-injection labels.

The results (Table 2) confirm the correctness of the error-generation process: BM25 reaches 94.4% agreement with the annotator, Dense Retrieval reaches 82.4%. The lower Dense Retrieval figure reflects that semantic retrieval errors are harder to classify unambiguously than lexical ones.

§4. The Metric Engine

The evaluation stack rests on an eleven-metric bundle computed for every trace. The first implementation contains 11 CPU-friendly metrics, selected to cover distinct RAG components while limiting redundant measurement. They are transparent, rule-based baselines that can later be complemented by multilingual embedding, Natural Language Inference (NLI), or LLM-as-a-judge methods, following the broader objective of RAG evaluation frameworks such as RAGAS [8], while emphasizing component-level evidence and predictable offline execution.

Stage	Metric	Definition and interpretation
Retrieval	Hit@K	1 if at least one gold chunk occurs in the first K retrieved chunks; otherwise 0.
Retrieval	Recall@K	Distinct gold chunks retrieved in the first K results, divided by the total number of gold chunks.
Retrieval	Precision@K	Gold matches divided by the actual number of returned chunks among the first K results.
Retrieval	MRR	Reciprocal rank of the first retrieved gold chunk; 0 when no gold chunk is retrieved.
Chunking	Gold Evidence Preservation	Proportion of gold evidence items marked as preserved and linked to an existing covering chunk.
Chunking	Chunk Integrity	One minus the proportion of chunks with structural defects (empty text, explicit truncation, invalid offsets, duplicate or missing identifiers).
Context	Evidence Coverage	Proportion of gold chunk identifiers included in the selected final context.
Context	Noise Ratio	Proportion of selected context chunks that are not gold evidence; lower is better.
Context	Context Truncation	Detects confirmed context truncation using explicit metadata or visible truncation markers.
Generation	Answer Relevance	Lexical overlap between valid query tokens and answer tokens, with language-aware tokenization and stop-word removal.
Generation	Faithfulness	Proportion of answer sentences receiving sufficient lexical support from the final context.

Table 3. Component-level metrics implemented in the Metric Engine.

Let G denote the set of gold evidence chunk identifiers, R_K the ordered set of the first K retrieved chunk identifiers, and C the set of selected context chunk identifiers. The retrieval metrics are

$$\text{Hit@}K = \mathbf{1}[G \cap R_K \neq \varnothing], \quad \text{Recall@}K = \frac{|G \cap R_K|}{|G|}, \quad \text{Precision@}K = \frac{|G \cap R_K|}{|R_K|}, \tag{1}$$

and MRR $= 1/r$, where r is the rank of the first retrieved gold chunk, or 0 if none is retrieved. These metrics require gold evidence identifiers and are skipped when such annotations are unavailable. Gold Evidence Preservation evaluates the chunking output rather than retrieval success: for every gold item it checks the preservation flag, the presence of covering chunk identifiers, and the existence of at least one referenced chunk. Chunk Integrity is a structural baseline: for N checked chunks and D chunks containing at least one detectable defect, integrity is $1-D/N$; its current implementation detects explicit truncation and malformed structure but does not claim to measure semantic coherence.

Evidence Coverage measures $|G \cap C|/|G|$, and Noise Ratio measures $|C \setminus G|/|C|$ and is the only first-version metric for which lower values indicate better quality. Context Truncation returns a confirmed score only when explicit truncation metadata or a visible marker is available; otherwise it is skipped rather than inferred from text length. The generation metrics are deliberately lightweight: Answer Relevance computes the proportion of valid query tokens shared with the answer after English- or Russian-aware tokenization and stop-word removal, and Faithfulness splits the answer into sentences and measures the fraction whose token overlap with the final context reaches a configurable support threshold. This score is an interpretable lexical-support baseline, not an NLI entailment score, and is therefore expected to miss paraphrases and subtle contradictions that require semantic reasoning.

Metric parameters are stored in a YAML configuration rather than embedded throughout the implementation. The evaluated configuration used $K = 5$, a chunk-integrity threshold of 0.8, noise thresholds of 0.2 and 0.5, a sentence-support threshold of 0.5, and faithfulness thresholds of 0.5 and 0.8; each result stores the effective parameters. The baseline is CPU-first with no required neural-model dependency. English and Russian token handling is implemented at the rule level and multilingual cases are covered by unit tests; however, the available complete experimental traces were English, so the reported full-batch and controlled-failure results should not be interpreted as empirical Russian-language validation. All first-version metrics are deterministic: repeated runs use identical inputs and configuration, the controlled-failure analysis uses a fixed random seed of 42 for bootstrap resampling, and runtime is recorded separately for every metric.

The Metric Engine was validated at three levels. First, unit and integration tests examined metric formulas, malformed fields, empty denominators, English and Russian token handling, input immutability, registry behavior, continuation after errors, JSONL output, and the complete execution of all 11 metrics through the batch runner (39 tests, all passing). Second, the engine was executed on two complete English trace sets—500 BM25 traces and 500 dense-retrieval traces—each trace producing 11 results, for 11,000 results with no execution errors; a repeated execution of the 500-trace evaluation was compared record by record and agreed exactly across all 5,500 compared results. Third, we conducted a paired controlled-failure sensitivity experiment: for each retriever, nine fault types were each represented by 20 post-hoc perturbations, paired with their healthy originals through original_trace_id, yielding $9 \times 20 \times 2 = 360$ paired records. The injected-fault label was removed before every metric call, so the engine could not exploit it as an input feature. For metric m and pair i, the paired change is $\Delta_{i,m} = \text{score}_{i,m}(\text{fault}) - \text{score}_{i,m}(\text{healthy})$ (for Noise Ratio the sign is reversed only for visualization, so that negative values consistently indicate degradation). We report the mean and median paired change, population standard deviation, a deterministic paired-bootstrap 95% confidence interval with 2,000 resamples, a standardized paired effect, and the paired rank-biserial effect. Thirty-eight fault–metric relationships were declared before analysis with an expected direction of change; 32 of them changed in the expected direction. This experiment evaluates component-level metric sensitivity to controlled trace perturbations, not full end-to-end fault propagation.

§5. Final Diagnosis Protocol and Controlled-Failure Evaluation

Each diagnosis is computed from the 11 MetricResult values attached to one trace_id, together with the trace metadata needed by the rule engine. The diagnosis procedure is deterministic and configuration-driven: configs/diagnostics_default.yaml fixes the required metrics, the stage order, the suppression relations, and the threshold values used by the engine:

$$\hat{y}_i = D(m_i, q_i, t_i; \theta), \tag{2}$$

where m_i is the component-level metric vector, q_i captures input quality and scope information, t_i denotes trace metadata, and θ collects the configured thresholds and precedence rules. D is a fixed decision function, not a statistical classifier.

The rule order follows an earliest-fault-first logic: input error and scope failure are checked before chunking, chunking before retrieval, retrieval before context, and context before generation (Figure 2). Suppression rules encode the fact that an upstream fault can explain a downstream symptom—for example, a chunking failure may suppress retrieval symptoms, so a retrieval drop is not interpreted as an independent root cause when the upstream evidence is already broken. This precedence matters because the metrics are not interchangeable: a low context.noise_ratio alone does not prove a context fault, a low generation.answer_relevance alone does not prove a generation fault if upstream evidence is already missing or truncated, and hit@k=0 does not by itself imply out_of_scope. The rule system therefore distinguishes a root cause from a downstream symptom and refuses to collapse the whole trace to its minimum metric value.

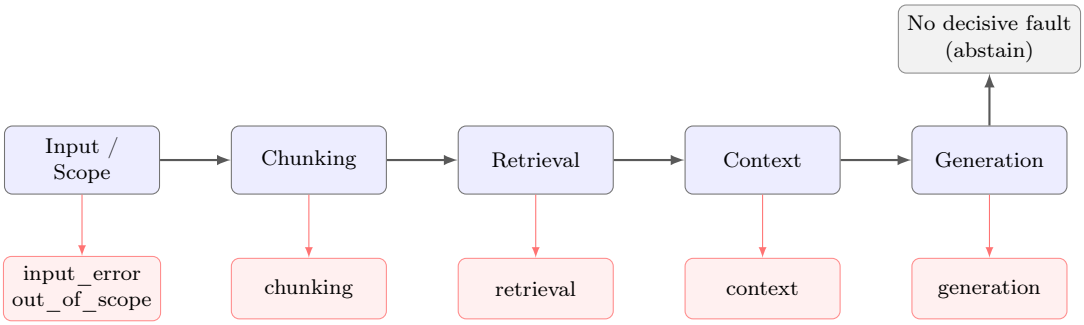


Figure 2. Earliest-fault-first diagnosis cascade. Each stage is checked in order; the first stage whose metric pattern fires assigns the fault family (downward). Suppression rules stop an already-explained downstream symptom from being blamed. If no stage fires decisively, the trace is routed to abstention rather than forced into a label.

At the pipeline level, the framework can aggregate the predicted fault family by stage:

$$\hat{p}_c^{(p)} = \frac{1}{N_p} \sum_{i:\,\text{pipeline}(i)=p} \mathbf{1}[\hat{y}_i = c]. \tag{3}$$

This aggregate is included as part of the framework rather than as a fully validated empirical claim; the current evidence supports trace-level diagnosis and controlled sensitivity analysis, not a complete pipeline-prevalence study over all possible production faults.

The English controlled-validation bundle contains 60 samples, split evenly across the two retrievers.

Retriever	Samples	Decisive	No-decision	Input err.	Strict acc.	Supported acc.	Coverage
Overall	60	47	12	1	51.67%	64.58%	78.33%
BM25	30	24	6	0	50.00%	62.50%	80.00%
Dense	30	23	7	0	53.33%	66.67%	76.67%

Table 4. English diagnosis summary on the controlled validation bundle. Supported exact accuracy is computed on the supported gold-label subset, not on the decisive-diagnosis count.

The diagnosis status distribution is 47 diagnosed, 12 no_decisive_fault, and 1 input_error. The supported-gold denominator differs from the decisive-diagnosis count: supported exact accuracy is computed on the supported gold-label subset {chunking, retrieval, generation}, containing 48 samples, even though the diagnosed count is 47 (Table 4). Strict exact accuracy is $31/60 = 51.67\%$, supported exact accuracy is $31/48 = 64.58\%$, coverage is $47/60 = 78.33\%$, and the abstention rate is $13/60 = 21.67\%$. BM25 and Dense remain close, and the small gap between them should not be overstated as a retriever ranking result—both stay within the same diagnostic regime.

Abstention (no_decisive_fault) is not a correct prediction; it is a conservative refusal to overcommit when evidence is insufficient or internally inconsistent. This lowers coverage but prevents the system from inventing a root cause the metrics do not support—a reliability feature rather than a failure of the evaluator.

The controlled-failure bundle is best read as a set of post-hoc component-level trace perturbations: it measures how the metrics respond when a fault is injected into an existing trace, not how a live pipeline would propagate a fault through a fresh downstream rerun.

Fault family	Stage	Primary signal	Observed change	Limitation
chunk_truncation	Chunking	chunk integrity	1.0 → 0.8	Does not cover all semantic corruption
missing_evidence	Retrieval	hit@k / recall@k	1.0 → 0.0	Faithfulness need not fall unless answer is regenerated
unsupported_answer	Generation	faithfulness	high → 0.0	Lexical proxy, not NLI
contradictory_answer	Generation	faithfulness	high → 0.0 / near 0	Cannot separate all semantic contradiction types
chunk_merge	Chunking	chunk integrity	no stable drop (dir. match = false)	Semantic fusion not reliably detected
distractor_context	Context	noise ratio	may increase	Old answer may be retained without regeneration

Table 5. Fault-level interpretation of the controlled perturbations, with the primary metric signal and the main limitation of the current rule set.

chunk_truncation gives the cleanest positive signal: chunking.chunk_integrity drops from 1.0 to 0.8 in both retrievers, a direct structural degradation signal that supports chunk-level root-cause attribution when the rest of the trace is consistent, though it does not prove sensitivity to all semantic forms of chunk damage. missing_evidence gives the strongest retrieval signature: retrieval.hit_at_k and retrieval.recall_at_k fall from 1.0 to 0.0, exactly the kind of pattern the rule cascade is designed to prioritize above downstream symptoms. unsupported_answer and contradictory_answer mainly move generation.faithfulness from a high value to 0.0 or near 0.0, supporting generation-level diagnosis when upstream evidence is coherent, though the metric remains a lexical support proxy rather than full semantic entailment (Table 5).

chunk_merge is the most useful negative result: its expected-direction agreement is false for the chunk-integrity signal, meaning the current metric is not yet sensitive to semantic fusion—a boundary condition on what the present rules can and cannot detect, not a cosmetic issue to be hidden. Similarly, distractor_context can raise context.noise_ratio, but if the answer is not regenerated after the perturbation, that change is a sensitivity result rather than a causal propagation result.

The broader uncertainty pattern is reflected in a 44-item review queue and a 32/38 expected-direction agreement over predeclared fault–metric relations: evidence that metrics react in the intended direction for most controlled perturbations, but not a diagnostic correctness score. The remaining mismatches are why the system retains abstention states and review cases rather than turning every perturbation into a hard label.

§6. Cross-Lingual Portability to Russian

A fault-diagnosis framework is only useful in practice if it transfers beyond the language and domain it was developed on. Our metrics and diagnosis logic were developed and validated on English data, so the key portability question is whether the entire framework—not just trace generation, but the controlled-failure evaluation that demonstrates fault localization—behaves the same way in a different language. We ran the pipeline and evaluation stack unchanged on Russian, so that any observed transfer is a property of the method rather than of language-specific tuning.

For trace generation, the existing Russian adapter processed 300 Russian samples through the identical chunking, BM25 retrieval, context-construction, and generation stages used for English, producing 300 complete, schema-valid Russian traces. For the controlled-failure evaluation, the healthy subset of these traces (gold evidence present in context) was extracted, the same nine controlled fault types were injected, and the same paired metric-sensitivity analysis that produced the English results was run, directed at a separate Russian data root so it neither overwrites nor contaminates the English artifacts. The only code change required was a small, backward-compatible --language option on the fault-injection script (defaulting to English, so English behavior is unchanged), because the original script hard-filtered to English-only traces.

Russian (BM25)	Value
Full traces generated	300
Generation success	300 / 300 (100%)
Retrieval Hit@5	98.7%
Retrieval Recall@5	77.2%
Gold in context	98.7%
Healthy traces (used for injection)	145

Table 6. Russian data-layer results, on par with the English figures in Table 1.

Table 6 confirms that the per-dataset adapters, multilingual chunking and retrieval, and the generator all transfer across languages with no pipeline change. For reference, on the full English set (500 traces) BM25 reached Hit@5 = 97.4% / Recall@5 = 78.0%, and the multilingual MiniLM dense retriever reached Hit@5 = 96.6% / Recall@5 = 77.4%—the two retrievers are effectively tied on this multi-hop data, with BM25 marginally ahead.

The audited Russian controlled-failure files show 360 paired records, 0 unmatched records, 11 metrics, and label isolation enforced before MetricRunner by removing injected_fault. This is enough to support a paired metric-sensitivity analysis and a portability check for the same fault families, but it is not, by itself, a full Russian trace-level diagnostic validation or a downstream re-execution study; the Russian bundle should be read as a cross-lingual supplement, not a second full diagnosis benchmark.

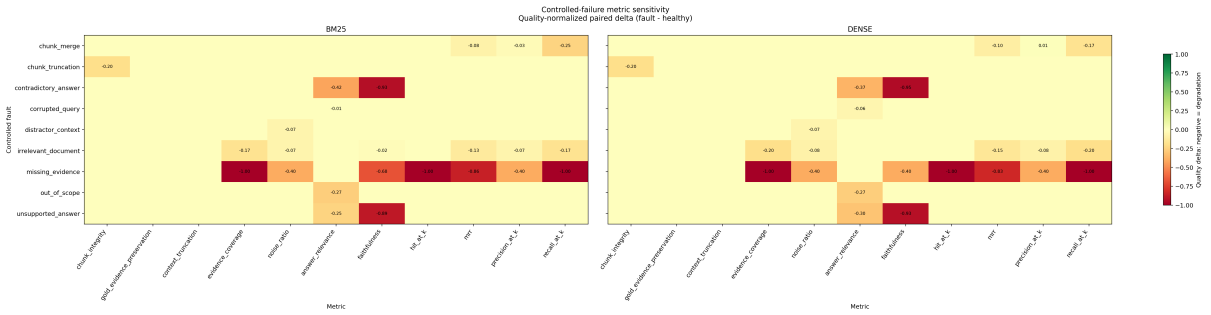


Figure 3. English dataset: controlled-failure metric sensitivity (BM25, Dense). Quality-normalized paired delta between fault-injected and healthy traces across fault types and evaluation metrics.

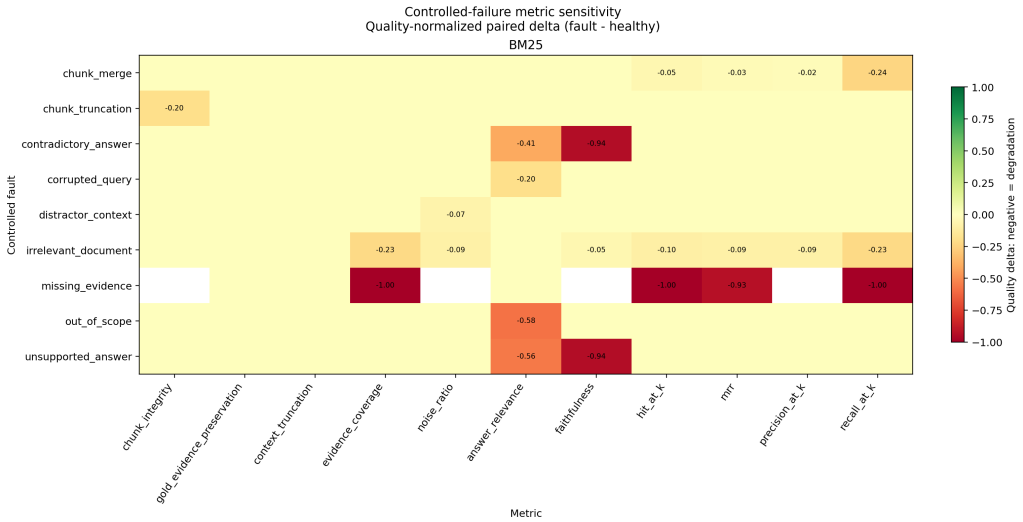


Figure 4. Russian dataset: controlled-failure metric sensitivity (BM25). Quality-normalized paired delta between fault-injected and healthy traces across fault types and evaluation metrics.

The visual comparison in Figures 3 and 4 confirms that the Russian controlled-failure sensitivity pattern closely reproduces the English one. The main fault families produce the same clear, component-isolated signals: missing_evidence collapses the retrieval metrics (hit_at_k, recall_at_k $\rightarrow -1.00$; mrr $\rightarrow -0.93$) without affecting generation metrics; contradictory_answer and unsupported_answer collapse faithfulness (≈ -0.94) without affecting retrieval metrics; and chunk_truncation moves only chunk_integrity.

Just as importantly, the same known metric gaps observed on English—chunk_merge not moving chunk_integrity, the unexercised context_truncation and gold_evidence_preservation columns, and the weak corrupted_query signal—reproduce identically on Russian, confirming they are method-level characteristics rather than language-specific artifacts.

Structural metrics remain relatively portable: chunk truncation still lowers chunk integrity, missing evidence still collapses retrieval, and the answer-level faithfulness proxy still falls for unsupported or contradictory answers. This suggests the framework transfers better on structure-sensitive signals than on purely lexical ones; lexical generation metrics (faithfulness, answer_relevance) are calibrated on English, so their absolute Russian values should be read as indicative rather than as a fully calibrated measurement, and the Russian section should not be used to claim universal multilingual generalization.

We report this result at its true level and no higher. The claim it supports is that the metric-sensitivity experiment reproduces the English diagnostic behavior on Russian, upgrading portability from the data layer to the evaluation layer. It remains a smaller-scale probe: the Russian healthy pool (145 traces) is smaller than the English one (293); as with English, faults are injected post hoc without re-running the downstream stages; and human-annotation validation was performed for English only, extending it to Russian being left as future work. In particular, the audited files do not show a full Russian downstream rerun of the generator or a fully calibrated multilingual judge. For that reason, the most defensible wording is that the Russian data extend the English findings as a cross-lingual supplement, not proof that the same thresholds are valid everywhere or that the method is ready for all languages and deployment settings.

§7. Discussion and Limitations

The strongest supported conclusion from this work is modest but useful: trace-level diagnosis is more interpretable than a single aggregate score, because it preserves the chain from fault family to metric signature to abstention behavior. The rule-based cascade makes that chain visible, which is why the controlled validation can separate decisive diagnoses from ambiguous cases instead of forcing every sample into a label.

That design comes with a clear coverage–accuracy trade-off. Coverage is deliberately lower than an unconditional classifier would obtain, because the engine abstains when evidence is too weak; in this setting that should be read as a reliability choice, not a hidden error. A conservative refusal to classify is preferable to a confident but unsupported root-cause claim.

The evidence also places hard limits on the current claims. The controlled failures are post-hoc component-level trace perturbations, so they verify metric sensitivity to direct field changes rather than true downstream re-execution through a live pipeline; the method does not yet prove how a damaged context would trigger a newly generated answer, and it does not yet close the gap between sensitivity and causal propagation. The same limitation explains why chunk_merge remains a known blind spot and why lexical faithfulness should not be overstated as a semantic judge. The Russian data strengthen the portability story, but only within the narrower cross-lingual supplement scope described; they do not remove the need for downstream re-execution, multilingual NLI or a calibrated LLM judge, human agreement studies, or pipeline-level prevalence validation.

Taken together, the framework can localize several important RAG fault families under controlled perturbation, it can do so with explicit abstention when evidence is weak, and its main failure modes are now visible rather than hidden. That is sufficient to support the bounded claims made in this paper, but not yet a full universal diagnosis claim.

§8. Conclusion

This paper combined four stages of a single project into one framework. We built a dataset-agnostic RAG Trace Platform that absorbs two incompatible data formats into a unified, schema-validated trace; validated trace quality and constructed a nine-fault controlled-failure benchmark with manual-annotation verification; defined an eleven-metric engine and a deterministic, precedence-based diagnosis rule cascade over it and evaluated the diagnosis on English, reaching 78.33% coverage at 64.58% supported accuracy

while explicitly documenting known blind spots; and reproduced both the trace-generation pipeline and the controlled-failure evaluation on Russian, finding that structure-sensitive fault signatures transfer while lexical metrics remain more language-dependent. Together, these results support a bounded but concrete claim: RAG fault localization by component-level metric diagnosis is portable across at least these two languages, is interpretable by design, and knows—and states—what it does not yet cover.

Author Contributions

This work was conducted as a collaborative research project within the SMILES 2026 summer school program. Figure 5 summarizes the division of responsibilities among the four team members.

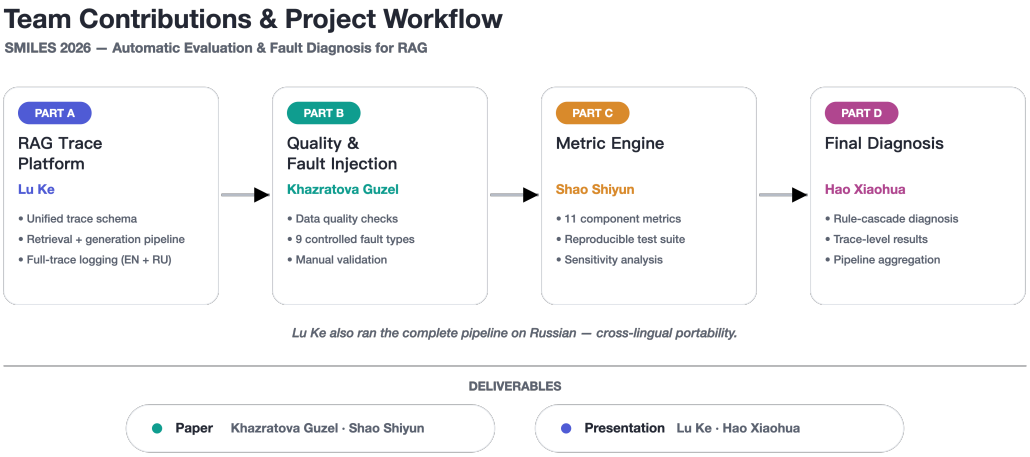


Figure 5. Team contributions and project workflow.

The project was supervised by Samsonova Darya, who served as the project mentor, providing methodological guidance, defining the overall research direction, and ensuring alignment with the project requirements throughout the SMILES 2026 summer school program. All authors contributed to the writing and revision of the manuscript, participated in regular team meetings, reviewed each other's code and documentation, and contributed to the final project presentation.

References

1. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, 2020.
2. Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2369–2380, Brussels, Belgium, 2018.
3. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 3982–3992, Hong Kong, China, 2019.
4. N. Reimers and I. Gurevych, "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 4512–4525, Online, 2020.
5. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.
6. V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 6769–6781, Online, 2020.
7. Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "GLM: General Language Model Pretraining with Autoregressive Blank Infilling," in Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 320–335, Dublin, Ireland, 2022.
8. S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in EACL Demo Track, 2024.